

Rôle : Networking

ADR 01 : Choix de la topologie réseau et haute disponibilité

1. Contexte

Pour déployer Nextcloud de manière professionnelle, nous devons garantir que l'application reste disponible même si un centre de données d'AWS tombe en panne. De plus, les instances applicatives et la base de données doivent être protégées des attaques directes depuis Internet.

2. Problème

Comment concevoir un réseau qui équilibre **haute disponibilité, sécurité maximale et maîtrise des coûts** ?

3. Décision

Nous avons décidé d'adopter une architecture **Multi-AZ (2 Zones de Disponibilité)** avec une segmentation en **6 sous-réseaux** et une sortie Internet contrôlée par **NAT Gateway**.

Les détails de l'arbitrage sont :

1. **Région** : eu-west-3 (Paris) pour la conformité RGPD.
2. **Zones** : a et b pour la redondance.
3. **NAT Gateway** : Une seule instance déployée dans le subnet public de l'AZ a.
4. **VPC Endpoints** : Utilisation de Endpoints S3 (Gateway) et Secrets Manager (Interface) pour garder le trafic sensible hors de l'Internet public.

4. Conséquences

Avantages (Pros) :

- **Résilience** : Si l'AZ a subit une panne, l'infrastructure dans l'AZ b prend le relais (Haute Disponibilité).
- **Sécurité** : Les serveurs Nextcloud et la base de données n'ont aucune adresse IP publique (Isolation totale).
- **Performance** : Le trafic vers S3 et Secrets Manager est acheminé via le backbone AWS (plus rapide et sécurisé).

Inconvénients / Risques (Cons) :

- **Coût** : La NAT Gateway et l'Endpoint Secrets Manager génèrent des coûts fixes (~40\$/mois).
- **Point de défaillance unique** : Si l'AZ a tombe, les instances de l'AZ b perdront leur accès Internet sortant (car la NAT Gateway est en AZ a), mais elles resteront accessibles par les utilisateurs via l'ALB.

5. Alternatives considérées

- **Instance NAT (EC2)** : Moins cher, mais nécessite une gestion manuelle des mises à jour et ne gère pas bien la montée en charge. Rejeté pour manque de fiabilité.
- **NAT Gateway par AZ (2 NATs)** : Idéal pour la résilience totale, mais rejeté car cela doublerait les coûts fixes pour ce TP.

Rôle : Compute

1. Contexte Pour déployer Nextcloud de manière professionnelle, nous devons garantir que l'application reste accessible même si le serveur applicatif tombe en panne. De plus, la connexion doit être sécurisée en HTTPS et le démarrage de Nextcloud doit être entièrement automatisé sans intervention manuelle.

2. Problème Comment déployer Nextcloud de façon résiliente, sécurisée en HTTPS et automatisée au démarrage sans clés AWS en dur ?

3. Décision Nous avons décidé d'utiliser un Application Load Balancer public HTTPS avec certificat auto-signé, couplé à un Auto Scaling Group maintenant une instance EC2 Amazon Linux 2023 qui lance Nextcloud en container Docker via un script `user_data`.

Les détails de l'arbitrage sont :

1. ALB : Load Balancer public avec listener HTTPS 443 et redirection automatique HTTP 80 vers HTTPS
2. Certificat : Auto-signé via provider Terraform TLS importé dans ACM
3. ASG : min=1 max=2 desired=1 pour l'auto-recovery uniquement
4. EC2 : Amazon Linux 2023 avec Docker, Nextcloud 30-apache lancé au boot via `user_data`
5. Sécurité : IMDSv2 obligatoire, credentials AWS via IAM Instance Profile uniquement

4. Conséquences

Avantages (Pros) :

- Résilience : Si l'EC2 crash, l'ASG en recrée une automatiquement sans intervention
- Sécurité : Aucune clé AWS en dur, IMDSv2 obligatoire contre les attaques SSRF
- Automatisation : Nextcloud démarre seul au boot, récupère ses secrets via Secrets Manager

Inconvénients / Risques (Cons) :

- Warning navigateur : Le certificat auto-signé génère un avertissement à accepter manuellement
- Pas de scaling horizontal : Une seule instance active à la fois car Nextcloud sans Redis ne tolère pas plusieurs frontaux simultanés sur le même stockage S3

5. Alternatives considérées

- ACM avec validation DNS : Plus propre en production mais nécessite un domaine Route53 validé. Rejeté car aucun domaine disponible pour ce TP.
- 2 instances actives derrière l'ALB : Rejeté car Nextcloud sans Redis cause des erreurs de file locking sur le stockage S3 partagé.
- Instance EC2 sans ASG : Rejeté car aucun mécanisme d'auto-recovery en cas de panne serveur.

Rôle : Data

1. Contexte Pour le cabinet d'avocats Kolab, la perte ou la fuite de dossiers clients sur Nextcloud est inacceptable. L'application a besoin d'une base de données robuste pour gérer ses utilisateurs/métadonnées, et d'un espace de stockage massif et sécurisé pour héberger les fichiers.

2. Problème Comment garantir la sécurité absolue des données (confidentialité), éviter toute perte accidentelle (durabilité) et maintenir le service en ligne lors d'une panne (haute disponibilité), tout en optimisant la facture de stockage ?

3. Décision Nous avons décidé de nous appuyer sur des services 100% managés par AWS : **Amazon RDS (PostgreSQL)** pour la base de données et **Amazon S3** pour les fichiers.

Les détails de l'arbitrage sont :

- **RDS Multi-AZ** : L'instance de base de données est répliquée de manière synchrone sur deux zones de disponibilité (a et b).
- **Sécurité au repos** : Chiffrement systématique via des clés KMS (SSE-KMS) pour le stockage RDS et les buckets S3.
- **S3 Versioning** : Activé sur le bucket de stockage principal (Primary) pour conserver l'historique des modifications.
- **S3 Lifecycle** : Création d'une règle sur le bucket de logs pour archiver les données vers Glacier après 30 jours et les supprimer à 90 jours.

4. Conséquences Avantages (Pros) : * Résilience (HA) : En cas de panne matérielle sur l'AZ principale, RDS bascule automatiquement sur l'AZ de secours en quelques minutes, sans intervention humaine.

- **Sécurité avancée** : Aucune donnée n'est stockée en clair sur les disques physiques. De plus, le mot de passe maître de la base est géré dynamiquement via Secrets Manager.
- **Durabilité** : Le versioning S3 agit comme un filet de sécurité imparable contre les erreurs de manipulation ou les attaques de type ransomware.

Inconvénients / Risques (Cons) : * Coût RDS : L'activation du Multi-AZ double littéralement la facture de la base de données (on paie deux instances t3.micro et le double de stockage SSD).

5. Alternatives considérées * Base de données montée sur EC2 : Plus économique sur le papier, mais oblige à gérer manuellement les patchs de l'OS, les sauvegardes et la configuration complexe de la haute disponibilité. Rejeté pour limiter la charge de maintenance opérationnelle (MCO).

- **Stockage Amazon EFS au lieu de S3** : S'utilise comme un disque réseau classique (NFS), ce qui est facile à configurer. Cependant, S3 est largement moins cher au

Gigaoctet et gère le versioning de manière beaucoup plus souple pour des millions de fichiers. Rejeté au profit de S3.

Rôle : Sécurité

1. Contexte

Dans le cadre du déploiement de Nextcloud, il est nécessaire de protéger les données sensibles (mots de passe, fichiers utilisateurs, base de données) ainsi que de sécuriser les accès aux ressources AWS.

L'infrastructure doit respecter les bonnes pratiques de sécurité, notamment le principe du moindre privilège et le chiffrement des données.

2. Problème

Comment sécuriser efficacement les accès aux ressources AWS et protéger les données sensibles tout en gardant une architecture simple et maintenable ?

3. Décision

Nous avons décidé d'utiliser une combinaison de services AWS managés pour assurer la sécurité : IAM, KMS, Secrets Manager et Security Groups.

Les détails de l'arbitrage sont :

- IAM Role : utilisé pour permettre aux instances EC2 d'accéder aux services AWS sans utiliser de credentials statiques
- KMS (Customer Managed Key) : utilisé pour chiffrer les données stockées (S3, RDS, Secrets Manager)
- Secrets Manager : stockage sécurisé des mots de passe (base de données et administrateur Nextcloud)
- Security Groups : mise en place de règles strictes entre ALB, application et base de données
- SSM Session Manager : accès aux instances EC2 sans ouvrir de port SSH

4. Conséquences

Avantages (Pros)

- Sécurité renforcée : aucune donnée sensible n'est stockée en clair
- Principe du moindre privilège respecté grâce à IAM
- Chiffrement des données via KMS
- Réduction de la surface d'attaque (pas d'accès direct aux instances ou à la base)
- Accès sécurisé aux instances sans SSH

Inconvénients / Risques (Cons)

- Complexité de configuration (IAM, politiques, KMS)
- Coût supplémentaire lié à KMS et Secrets Manager (~2-3€/mois)
- Dépendance aux services managés AWS

5. Alternatives considérées

- Stocker les mots de passe directement dans Terraform
→ Rejeté car non sécurisé (exposition dans le code et le state)
- Utiliser des clés d'accès IAM statiques
→ Rejeté car moins sécurisé que les rôles IAM
- Ne pas utiliser de chiffrement KMS
→ Rejeté car ne respecte pas les bonnes pratiques de sécurité